HACKATHON PROJECT

InRetail

Smart Inventory Management System

A database-driven platform to track products, stock movement, suppliers and customer orders — all in one operational view.

2520030003-K.BHAVANA
2520030032-A.AMULYA
2520030124-M.NIHAL

SEC-7

Presented by: [Team Name] | [Team Members]

01 / THE CHALLENGE

Inventory decisions shouldn't rely on guesswork

Disconnected records and delayed stock updates create avoidable operational friction.

Stock blind spots

Teams may not know which products are running low until a sale or fulfilment is affected.

Scattered information

Product, supplier, order and stock records are difficult to reconcile when stored separately.

Slow decisions

Without a clear view of inventory value and stock movement, replenishment becomes reactive.

THE OPPORTUNITY

Create one reliable source of truth for inventory operations — with actionable low-stock visibility.

One workspace. Four connected data domains.

InRetail combines a responsive dashboard with REST APIs and MySQL persistence.

PRODUCTS

SKU • category • price • stock

SUPPLIERS

Contacts • email • phone

ORDERS

Order number • customer • status

STOCK MOVEMENTS

IN / OUT • quantity • timestamp

The dashboard turns these records into operational signals: revenue, order count, active products and items needing attention.

A straightforward, maintainable three-layer design

The browser talks to Spring Boot APIs; Spring Data JPA persists the domain model in MySQL.



API examples

GET /api/dashboard • GET /api/products • POST /api/products/{id}/stock • GET /api/movements

A relational model built around inventory operations

Four core tables keep the main business entities clear and support traceable stock updates.

products

PK id • UNIQUE sku
name, category, current_stock
reorder_point, unit_price, expiry_date

suppliers

PK id
name, contact_person
email, phone, address

1 product → many stock movements

customer_orders

PK id
order_number, customer_name
total_amount, status, order_date

stock_movements

PK id • FK product_id → products.id
movement_type, quantity, note
created_at

Every stock change leaves a trail

Stock IN and OUT operations update the product balance and create a movement record.

01

Select product

Identify the SKU to update.

02

Choose movement

Record IN or OUT and quantity.

03

Validate balance

Reject zero/negative quantities and insufficient stock.

04

Persist history

Save updated stock and movement details.

Business rule: stock cannot go below zero; each accepted adjustment is auditable.

From raw records to useful operational signals

A focused dashboard supports day-to-day monitoring and core inventory workflows.

At-a-glance KPIs

Revenue • orders • low stock • out of stock • active products

Product management

Create, search, update and delete product records

Stock control

Receive stock, issue stock and inspect recent movement history

Supplier & order views

Review supplier contacts and customer order statuses

Inventory reporting

Inventory value, product counts and stock-status summaries

Attention queue

Surface products at or below their configured reorder point

Note: dashboard totals are database-backed; the monthly revenue trend shown in the starter UI is illustrative.

A 90-second story judges can follow

Show the database-backed workflow end to end rather than clicking through every screen.

1	**Open dashboard**	Point out active products and low/out-of-stock alerts.
2	**Find a product**	Search by product name or SKU.
3	**Receive stock**	Add an IN movement and show the updated balance.
4	**Issue stock**	Try an OUT movement; demonstrate insufficient-stock validation.
5	**Show audit trail**	Open recent movements and explain product_id relationship.

Demo setup: Java 17+ • Maven 3.8+ • MySQL 8+ • configure DB credentials • mvn spring-boot:run

A strong foundation for smarter retail operations

The current build focuses on dependable inventory records and workflows. These are logical next increments.

01 **Sales line items**

Power true time-series revenue and product-level sales analytics.

02 **Demand forecasting**

Use dated sales history to estimate replenishment needs.

03 **Expiry & alerts**

Add expiry monitoring, configurable thresholds and notifications.

04 **Access & audit**

Introduce role-based access, user attribution and stronger audit controls.

InRetail makes inventory visible, changes traceable and replenishment decisions easier to act on.

THANK YOU • QUESTIONS?